PROJECT PLAN

Banking Transaction System

Project information	Details
Project type	Java-based web application with graphical user interface
Estimated duration	8 weeks (approximately 2 months)
Team size	5 members
Delivery approach	Agile-inspired, four 2-week iterations
Document status	Proposed baseline; dates and technology choices subject to team approval

1. Project Overview

The Banking Transaction System is a Java-based web application for managing bank accounts and processing deposits and withdrawals. It will display transaction history and persist data using a relational database. The project emphasizes object-oriented design, Java collections where appropriate, exception handling, static reviews, manual testing, automated UI testing, and defect/test activity tracking in Jira.

2. Objectives

- Implement correct account creation, balance enquiry, deposits, withdrawals, and transaction history.
- Validate inputs and reject invalid transactions without changing account balances.
- Use Java OOP, exception handling, and persistent file/database storage (database selected for this plan).
- Document test scenarios, test cases, execution evidence, defects, and static review findings.
- Automate key browser workflows with Selenium WebDriver and test backend APIs using Postman where endpoints are implemented.

3. Scope and Boundaries

In scope	Out of scope / assumptions
Account creation and account lookup; balance display	No integration with real banks or payment networks
Deposits, withdrawals, transaction history	No real-money movement or production banking deployment
Input validation, insufficient-funds handling, persistence	No complex transfers, loans, or interest calculations unless approved
Basic browser UI and REST endpoints for core operations	Authentication/authorization requirements to be confirmed before design
Manual tests, static reviews, Selenium UI tests, Jira reports	Postman tests apply to implemented API endpoints

4. Implementation Methodology

The team will work in four two-week iterations with a short planning meeting at each iteration start, regular progress check-ins, and a review/demo at each iteration end.

Development and testing will overlap: test design starts during requirements/design, and unit/manual checks run as features become available. GitHub will support version control; Jira will track tasks, test activities, and defects.

5. Work Breakdown and Deliverables

Phase / weeks	Key activities	Expected output
1. Requirements & setup / W1	Confirm use cases, acceptance criteria, architecture assumptions; set up repository and Jira.	Requirements baseline; project documents; initial backlog
2. Design / W2	UML/class design, database schema, UI wireframes, API contracts; draft test scenarios.	Design package; test scenarios; review notes
3. Core development / W3–W4	Implement Java models/services, persistence, validation, REST endpoints and essential web screens; write unit tests.	Working core banking features; initial unit tests
4. Functional testing & stabilization / W5	Execute manual cases, negative/ boundary tests, static reviews; log and prioritize defects; fix critical issues.	Manual test evidence; defect log; static review report
5. UI/API automation / W6–W7	Automate critical UI flows with Selenium; API checks with Postman; rerun regression and fix defects.	Automated suites; API collection; execution reports
6. Final validation & delivery / W8	End-to-end regression, resolve blocking defects, consolidate Jira metrics, documentation and demo.	Application, test reports, source code and final presentation

6. Eight-Week Timeline

Workstream	W1	W2	W3	W4	W5	W6	W7	W8
Requirements / setup								
Design / test design								
Development / unit tests								
Manual testing / defect fixes								
UI and API automation								
Regression / delivery								

Milestone checkpoints: end W2—approved design and requirements; end W4—demonstrable core flows; end W6—first automated checks; end W8—final tested release. Timing is indicative and may change after the first milestone review.

7. Proposed Technology Stack

Area	Proposed tools	Reason
Backend	Java + Spring Boot	Core business logic and REST API

Frontend	HTML, CSS, JavaScript (server-rendered templates optional)	Browser-accessible GUI for Selenium
Unit testing	JUnit	Java logic validation
UI automation	Selenium WebDriver	Browser workflow testing
API testing	Postman	REST endpoint checks
Tracking / source	Jira / GitHub	Defects, activities, and version control

8. Quality and Acceptance Criteria

Core use cases pass: account creation, valid deposit, valid withdrawal, balance enquiry, and transaction history.

Invalid/negative/zero amounts and insufficient funds are rejected with clear feedback; rejected operations do not alter stored balances.

Transaction data remains consistent after application restart; money amounts use a decimal-safe representation (e.g., BigDecimal), not floating-point arithmetic.

Critical defects are resolved before final submission; remaining known limitations are documented.

Manual test cases, Selenium evidence, static review notes, and Jira activity reports are included in the handover.

9. Final Deliverables

Java application with account and transaction classes, web UI, and persistence.

Documented validation and exception handling for invalid operations.

Manual test plan, test cases, execution results, and static review/Jira feedback.

Selenium WebDriver scripts for UI workflows.

Postman API tests for available backend endpoints.

Jira test activity and defect reports, source repository, setup instructions, and final presentation.